Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Norte - Pau dos Ferros

Análise e Desenvolvimento de Sistemas - Semestre: 2026.2

Disciplina: Paradigmas de Linguagens de Programação

Exercícios - Clojure

Nos exercícios abaixo, você vai definir **funções** com `defn`. Em Clojure não declaramos tipos; por isso, escreva uma *docstring* ou um comentário dizendo o que a função recebe e devolve (veja o quickstart).

Parte 1 - Definindo novas funções

1. Crie uma função `sum-squares` que receba dois números x e y e calcule a soma dos seus quadrados.
2. Defina a função `circle-area` que receba um raio r e calcule a área de um círculo com esse raio, dada por pi vezes o raio ao quadrado. Dica: Clojure usa a constante do Java `Math/PI`.
3. Defina uma função `age` que receba o ano de nascimento de uma pessoa e o ano atual, produzindo como resultado a idade (aproximada) da pessoa.
4. Defina uma função `elderly?` que receba uma idade e resulte verdadeiro caso a idade seja maior que 65 anos. (Por convenção, funções que devolvem `true`/`false` terminam com `?`.)
5. Defina uma função `html-item` que receba uma string e adicione tags `<li>` e `</li>` como prefixo e sufixo, respectivamente. Por exemplo, se a entrada for `"abc"`, a saída será `"<li>abc</li>"`. Use a função `str` para concatenar strings.
6. Crie uma função `starts-with-a?` que receba uma string e verifique se ela inicia com o caracter `\A`. Dica: teste antes a função pré-definida `first`.
7. Defina uma função `verb?` que receba uma string e verifique se ela termina com o caracter `\r`. Antes desse exercício, teste no REPL a função pré-definida `last`, que retorna o último elemento de uma coleção. Dica: conheça também a Clojure Cheat Sheet, que reúne as funções mais usadas.
8. Crie uma função `vowel?` que receba um caracter e verifique se ele é uma vogal minúscula.
9. Crie uma função `eq-heads?` que verifique se 2 coleções possuem o mesmo primeiro elemento. Use a função `first` e a função `=` para verificar igualdade.
10. Em Clojure, um **conjunto** (`#{...}`) pode ser usado para testar pertencimento com `contains?`. Teste no REPL:

None

```
    (contains? #{1 2 3} 3)
    (contains? #{1 2 3} 4)
    (contains? #{\a \b} \a)
    (contains? [1 2 3] 3)
    (some #{\c} "abcd")
    (some #{\A} "abcd")
```

None

```
Repare no resultado de `(contains? [1 2 3] 3)`: em vetores,
`contains?` olha os **índices**, não os valores! Agora use um
conjunto para criar uma função `vowel-2?` que verifique se um
caracter é uma vogal, tanto maiúscula como minúscula.
```

Parte 2 - Começando a usar funções de alta ordem

Para fazer os exercícios abaixo, você vai usar as **funções de alta ordem** `map` e `filter`, que foram descritas na introdução a funções em Clojure (veja os slides da disciplina). Lembre-se de que elas devolvem *sequências*; para transformar uma sequência de caracteres em string, use `(apply str ...)`.

Muitos desses exercícios também vão usar funções que você definiu na parte anterior desta prática (procure lembrar delas). Sempre que possível, reuse funções já definidas. Quando não houver uma função já definida, você deverá criá-la.

1. Crie uma função `itemize` que receba uma lista de nomes e aplique a função `html-item` em cada nome.
2. Crie uma função `only-vowels` que receba uma string e retorne outra contendo somente suas vogais. Por exemplo: `(only-vowels "abracadabra")` vai retornar `"aaaaa"`.
3. Escreva uma função `only-elderly` que, dada uma lista de idades, selecione somente as que forem maiores que 65 anos.
4. Crie uma função `only-long-words` que receba uma lista de strings e retorne somente as strings longas (use a função `long-word?` definida no código de exemplo no início da prática).
5. Escreva uma função `only-even` que receba uma lista de números inteiros e retorne somente aqueles que forem pares.
6. Escreva uma função `only-between-60-and-80` que receba uma lista de números e retorne somente os que estiverem entre 60 e 80, inclusive. Você deverá criar uma função auxiliar `between-60-and-80?` e usar `and` para expressar o operador "E" lógico em Clojure.
7. Crie uma função `count-spaces` que receba uma string e retorne o número de espaços nela contidos. Dica 1: você vai precisar de uma função que identifica

espaços (lembre-se de `\space`). Dica 2: aplique funções consecutivamente, isto é, use o resultado de uma função como argumento para outra.

8. Escreva uma função `calc-areas` que, dada uma lista de valores de raios de círculos, retorne uma lista com a área correspondente a cada raio.
9. Neste exercício, você vai criar uma função `char-found?`, que verifica se um caracter (primeiro argumento) está contido em uma string (segundo argumento), mas usando uma função de alta ordem. Exemplos de uso da função:

None

```
user=> (char-found? \a "")
false
user=> (char-found? \a "uau")
true
```